Homework 11

Question 1: Implementing K-means

Part (a): One iteration

Random seed: **43211**.

```
set.seed(43211)
x <- matrix(rnorm(10 * 2), nrow = 10, ncol = 2)
colnames(x) <- c("x1", "x2")

# Draw random labels again if either cluster is empty.
initial_cluster <- sample(1:2, nrow(x), replace = TRUE)
while (length(unique(initial_cluster)) < 2) {
  initial_cluster <- sample(1:2, nrow(x), replace = TRUE)
}
```

```
centers <- rbind(
  colMeans(x[initial_cluster == 1, , drop = FALSE]),
  colMeans(x[initial_cluster == 2, , drop = FALSE])
)
rownames(centers) <- c("Cluster 1", "Cluster 2")

distances <- sapply(1:2, function(k) {
  differences <- sweep(x, 2, centers[k, ], FUN = "-")
  sqrt(rowSums(differences^2))
})
updated_cluster <- max.col(-distances, ties.method = "first")

knitr::kable(centers, digits = 4, caption = "Initial cluster means")
```

Table 1: Initial cluster means

	x1	x2
Cluster 1	0.5732	0.0351
Cluster 2	-0.0354	-0.3897

```
knitr::kable(
  data.frame(observation = 1:nrow(x), x,
             initial_cluster, updated_cluster),
  digits = 4, caption = "Observations and cluster assignments"
)
```

Table 2: Observations and cluster assignments

observation	x1	x2	initial_cluster	updated_cluster
1	0.3928	0.3908	2	1
2	0.8630	-0.8521	1	1
3	0.8142	-0.3796	1	1
4	1.7948	3.2492	1	1
5	-0.7928	-0.3298	1	2
6	0.6744	0.5857	1	1
7	0.0856	-2.0627	1	2
8	-0.4366	0.0990	2	2
9	-0.5292	-1.2755	2	2
10	0.4315	-0.7732	2	2

```
cluster_colors <- c("#0072B2", "#D55E00")
plot(x, col = cluster_colors[updated_cluster], pch = 19,
     xlab = "Coordinate 1", ylab = "Coordinate 2", asp = 1,
     main = "K-means: one iteration")
text(x, labels = 1:nrow(x), pos = 3, cex = 0.8)
points(centers, col = cluster_colors, pch = 4, cex = 2, lwd = 3)
legend("topleft", legend = c("Cluster 1", "Cluster 2"),
       col = cluster_colors, pch = 19, bty = "n")
```

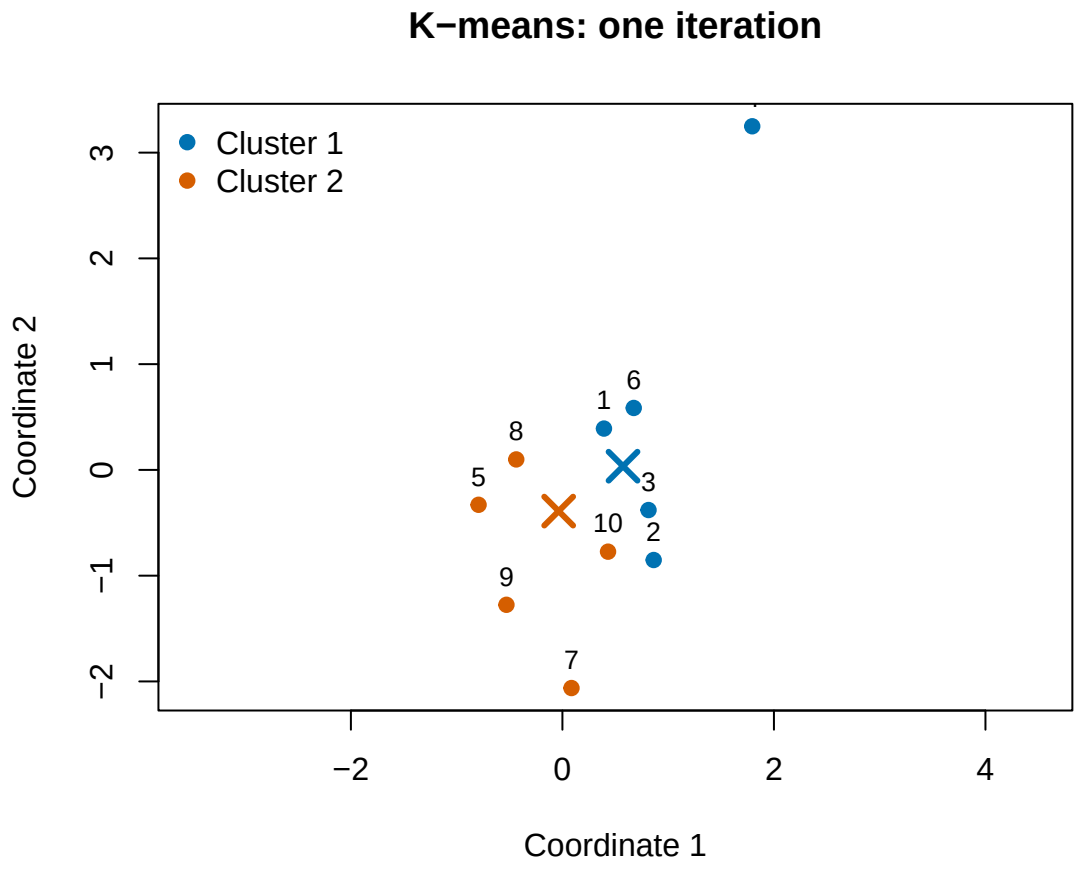


Figure 1: Assignments after one iteration. Crosses mark the initial cluster means used to assign the points.

Part (b): Repeating the algorithm

```r
my_kmeans <- function(x, K, initial = NULL, max_iter = 100L) {
  x <- as.matrix(x)
  n <- nrow(x)
  p <- ncol(x)
  if (!is.numeric(x) || n < 1L || p < 1L || any(!is.finite(x))) {
    stop("x must be a nonempty, finite numeric matrix.")
  }
  if (length(K) != 1L || !is.finite(K) || K != floor(K) ||
      K < 1L || K > n) stop("K must be an integer between 1 and n.")
  if (length(max_iter) != 1L || !is.finite(max_iter) ||
      max_iter != floor(max_iter) || max_iter < 1L) {
    stop("max_iter must be a positive integer.")
  }

  # Include every label at least once, then randomly shuffle the labels.
  if (is.null(initial)) {
    initial <- sample(c(seq_len(K),
                        sample.int(K, n - K, replace = TRUE)))
  }
  if (length(initial) != n || anyNA(initial) ||
      !all(initial %in% seq_len(K)) || length(unique(initial)) != K) {
    stop("initial must assign every observation and include every cluster.")
  }
  cluster <- as.integer(initial)
  centers <- matrix(NA_real_, nrow = K, ncol = p,
                    dimnames = list(paste("Cluster", seq_len(K)), colnames(x)))

  update_means <- function(cluster, centers) {
    for (k in seq_len(K)) {
      members <- cluster == k
      if (any(members)) {
        centers[k, ] <- colMeans(x[members, , drop = FALSE])
      }
    }
    centers
  }

  converged <- FALSE
  for (iteration in seq_len(max_iter)) {
    centers <- update_means(cluster, centers)
    squared_distances <- matrix(0, nrow = n, ncol = K)
    for (k in seq_len(K)) {
      squared_distances[, k] <- rowSums(
        sweep(x, 2, centers[k, ], FUN = "-")^2
      )
    }
    next_cluster <- max.col(-squared_distances, ties.method = "first")
    if (identical(next_cluster, cluster)) {
      converged <- TRUE
      break
    }
    cluster <- next_cluster
  }

  # Also update means if the iteration limit ended the loop.
```

```
  centers <- update_means(cluster, centers)
  wcss <- sum((x - centers[cluster, , drop = FALSE])^2)
  list(cluster = cluster, centers = centers, wcss = wcss,
       iterations = iteration, converged = converged)
}
```

Part (c): Comparing two starts

Run 1 uses the initial assignments from part (a); run 2 uses seed **43212**. Both use the same 10 observations.

```
set.seed(43212)
initial_cluster2 <- sample(1:2, nrow(x), replace = TRUE)
while (length(unique(initial_cluster2)) < 2 ||
       identical(initial_cluster2, initial_cluster)) {
  initial_cluster2 <- sample(1:2, nrow(x), replace = TRUE)
}
fit1 <- my_kmeans(x, K = 2, initial = initial_cluster)
fit2 <- my_kmeans(x, K = 2, initial = initial_cluster2)

knitr::kable(data.frame(
  observation = seq_len(nrow(x)),
  initial_run1 = initial_cluster, final_run1 = fit1$cluster,
  initial_run2 = initial_cluster2, final_run2 = fit2$cluster
), caption = "Initial and final assignments for both runs")
```

Table 3: Initial and final assignments for both runs

observation	initial_run1	final_run1	initial_run2	final_run2
1	2	1	2	2
2	1	2	2	1
3	1	2	2	1
4	1	1	2	2
5	1	2	1	1
6	1	1	1	2
7	1	2	2	1
8	2	2	1	1
9	2	2	2	1
10	2	2	1	1

```
knitr::kable(rbind(
  data.frame(run = 1, cluster = 1:2, fit1$centers, row.names = NULL),
  data.frame(run = 2, cluster = 1:2, fit2$centers, row.names = NULL)
), digits = 4, caption = "Final cluster means")
```

Table 4: Final cluster means

run	cluster	x1	x2
1	1	0.9540	1.4086
1	2	0.0622	-0.7963
2	1	0.0622	-0.7963
2	2	0.9540	1.4086

```
knitr::kable(data.frame(
  run = 1:2, WCSS = c(fit1$wcss, fit2$wcss),
  iterations = c(fit1$iterations, fit2$iterations),
  converged = c(fit1$converged, fit2$converged)
), digits = 4, caption = "Objective values and convergence")
```

Table 5: Objective values and convergence

run	WCSS	iterations	converged
1	11.9039	4	TRUE
2	11.9039	5	TRUE

```
# Compare pairwise membership, because cluster labels are arbitrary.
same_grouping <- identical(outer(fit1$cluster, fit1$cluster, "=="),
                           outer(fit2$cluster, fit2$cluster, "=="))
same_objective <- isTRUE(all.equal(fit1$wcss, fit2$wcss))
```

```
old_par <- par(mfrow = c(1, 2))
for (run in 1:2) {
  fit <- list(fit1, fit2)[[run]]
  plot(x, col = cluster_colors[fit$cluster], pch = 19, asp = 1,
       xlab = "Coordinate 1", ylab = "Coordinate 2",
       main = paste0("Run ", run, ": WCSS = ", round(fit$wcss, 4)))
  text(x, labels = seq_len(nrow(x)), pos = 3, cex = 0.8)
  points(fit$centers, col = cluster_colors, pch = 4, cex = 2, lwd = 3)
  legend("topleft", legend = c("Cluster 1", "Cluster 2"),
         col = cluster_colors, pch = 19, bty = "n", cex = 0.8)
}
par(old_par)
```

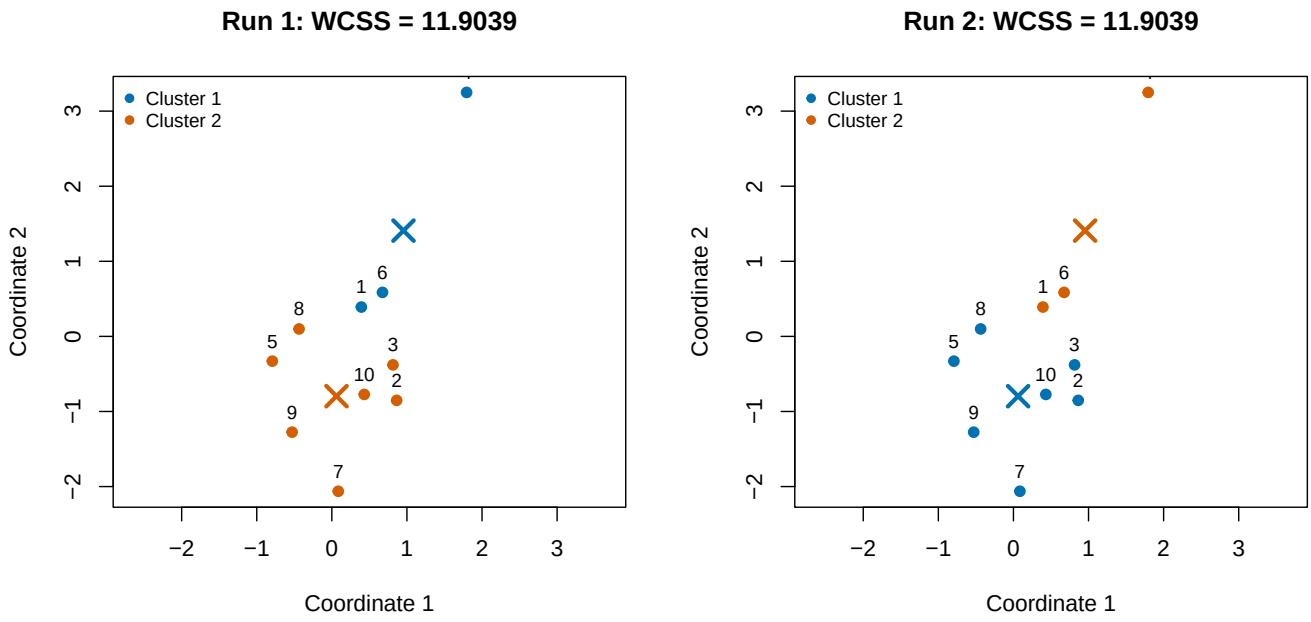


Figure 2: Final groups from two random initial assignments; crosses mark final cluster means. Both panels use the same axes.

Both runs produce the same groups: **{1, 4, 6}** and **{2, 3, 5, 7, 8, 9, 10}**, with WCSS **11.9039**. The cluster labels are reversed, but this does not change the grouping.

The two starts converge to the same solution here. In general, different initial assignments can lead to different local minima, so agreement does not establish a global minimum.

Question 2: Image color compression

Part (a): Image and RGB matrix

Image: `IMG_4480.HEIC`. Original dimensions: **4032 × 3024 pixels** (width × height); resized dimensions: **200 × 150 pixels**.

```
knitr::include_graphics("data/img4480-200.png")
```



Figure 3: Original colors of IMG_4480.HEIC, resized to 200 by 150 pixels before clustering.

RGB values are on a **0–1 scale** for all fits.

```
image_width <- 200L
image_height <- 150L
pixels <- as.matrix(read.csv("data/img4480-rgb.csv")) / 255
stopifnot(identical(dim(pixels), c(image_width * image_height, 3L)),
          all(pixels >= 0 & pixels <= 1))
dim(pixels)
```

```
[1] 30000     3
```

The data matrix has **30,000 rows and 3 columns**: one observation per pixel and one variable per color channel.

Part (b): Cluster and reconstruct

For each K, the reported fit has the smallest WCSS among three random starts.

```
Ks <- c(2L, 3L, 5L, 10L)
image_fits <- lapply(Ks, function(K) {
  candidates <- lapply(1:3, function(start) {
    set.seed(43220 + 10 * K + start)
    my_kmeans(pixels, K = K, max_iter = 500L)
  })
  stopifnot(all(vapply(candidates, function(f) f$converged, logical(1))))
  best <- which.min(vapply(candidates, function(f) f$wcss, numeric(1)))
  fit <- candidates[[best]]
  fit$seed <- 43220 + 10 * K + best
  fit
})
image_summary <- data.frame(
  K = Ks,
  seed = vapply(image_fits, function(f) f$seed, numeric(1)),
  iterations = vapply(image_fits, function(f) f$iterations, integer(1)),
  WCSS = vapply(image_fits, function(f) f$wcss, numeric(1))
)
knitr::kable(image_summary, digits = 4,
             caption = "Within-cluster sums of squares on the 0-1 RGB scale")
```

Table 6: Within-cluster sums of squares on the 0-1 RGB scale

K	seed	iterations	WCSS
2	43241	10	1225.1676
3	43251	29	746.8870
5	43273	84	418.0881
10	43322	81	182.2252

```
image_means <- do.call(rbind, lapply(seq_along(Ks), function(j) {
  data.frame(K = Ks[j], cluster = seq_len(Ks[j]),
             image_fits[[j]]$centers, row.names = NULL)
}))
knitr::kable(image_means, digits = 4, caption = "Cluster means (RGB palette)")
```

Table 7: Cluster means (RGB palette)

K	cluster	red	green	blue
2	1	0.4298	0.5750	0.8098
2	2	0.2757	0.3074	0.2945
3	1	0.1902	0.2232	0.2178
3	2	0.4252	0.5864	0.8599
3	3	0.4005	0.4408	0.4376
5	1	0.1247	0.1531	0.1410
5	2	0.2905	0.3280	0.3270
5	3	0.3554	0.5223	0.8254
5	4	0.4690	0.4977	0.4759
5	5	0.5147	0.6730	0.9008
10	1	0.7364	0.7517	0.7599
10	2	0.4046	0.5832	0.8897

K	cluster	red	green	blue
10	3	0.3332	0.4955	0.8143
10	4	0.3336	0.4585	0.5915
10	5	0.5679	0.5729	0.5210
10	6	0.0860	0.1134	0.1000
10	7	0.5074	0.6845	0.9300
10	8	0.2911	0.3333	0.3442
10	9	0.4305	0.4415	0.3807
10	10	0.2103	0.2418	0.2338

```
draw_pixels <- function(pixel_matrix, title) {
  colors <- rgb(pixel_matrix[, 1], pixel_matrix[, 2], pixel_matrix[, 3])
  raster <- as.raster(matrix(colors, nrow = image_height,
                             ncol = image_width, byrow = TRUE))
  plot.new()
  plot.window(xlim = c(0, image_width), ylim = c(0, image_height),
              asp = 1, xaxs = "i", yaxs = "i")
  rasterImage(raster, 0, 0, image_width, image_height, interpolate = FALSE)
  title(main = title)
}
old_par <- par(mfrow = c(2, 2), mar = c(0.5, 0.5, 2.5, 0.5))
for (j in seq_along(Ks)) {
  fit <- image_fits[[j]]
  reconstructed <- fit$centers[fit$cluster, , drop = FALSE]
  draw_pixels(reconstructed,
              paste0("K = ", Ks[j], "; WCSS = ", round(fit$wcss, 2)))
}
par(old_par)
```

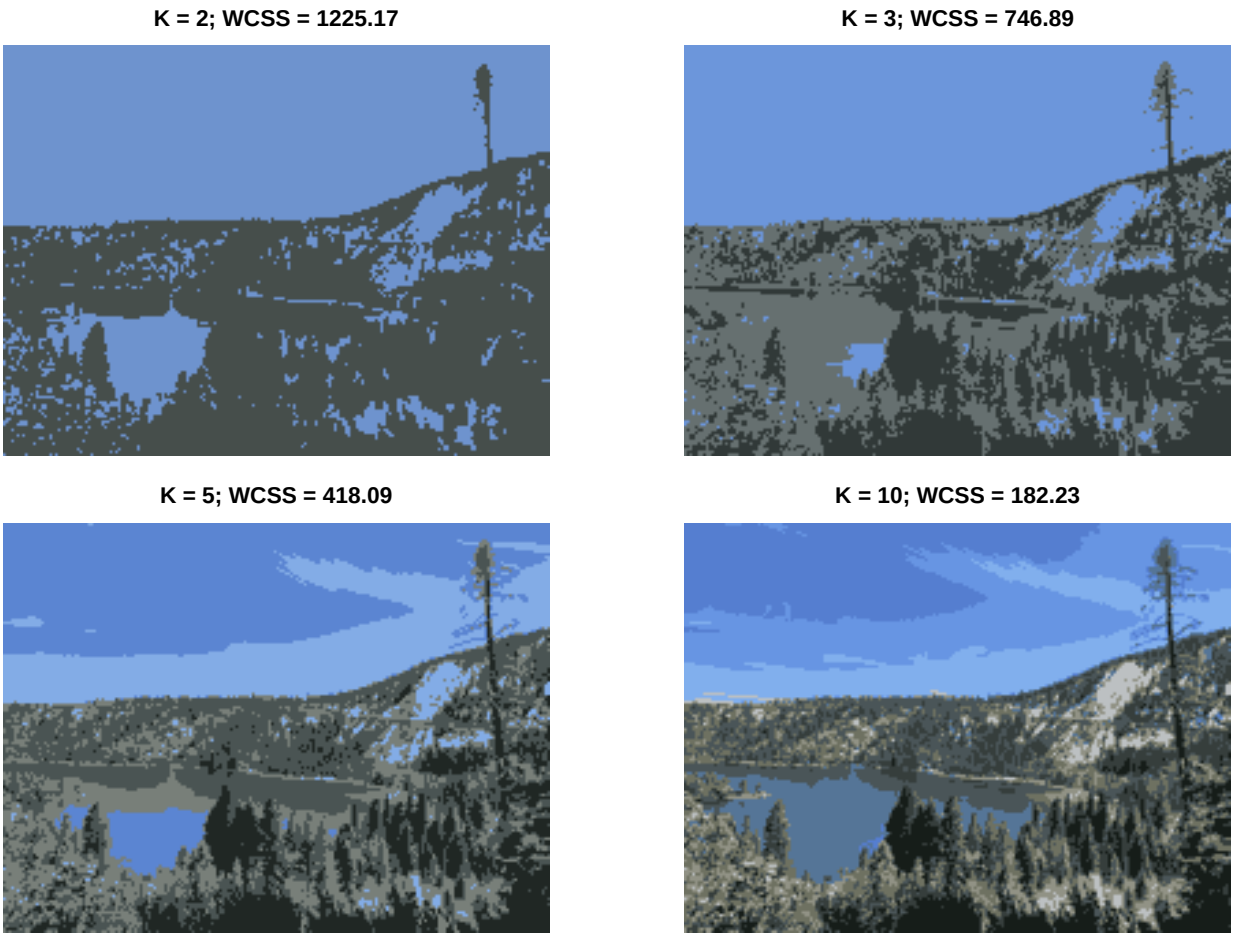


Figure 4: The four reconstructions use 2, 3, 5, and 10 colors, respectively.

Part (c): Choosing K

I would choose $K = 10$. Two or three colors lose much of the forest and mountain detail. Five colors improve the sky and trees, but ten better distinguish the lake, bright mountain areas, and forest. Increasing K from 5 to 10 reduces WCSS from **418.0881** to **182.2252**, a **56.4%** reduction. This visible improvement makes ten colors worth using, even though some color banding remains.

Question 3: Hierarchical clustering of cars

Part (a): Standardization and dendrograms

Data source: the supplied `data/mtcars.csv` (32 cars, 11 numerical variables).

```
cars <- read.csv("data/mtcars.csv", check.names = FALSE)
car_data <- cars[, names(cars) != "car", drop = FALSE]
rownames(car_data) <- cars$car
stopifnot(nrow(car_data) == 32L, ncol(car_data) == 11L,
          all(vapply(car_data, is.numeric, logical(1))),
          all(is.finite(as.matrix(car_data))))
car_z <- scale(car_data)
stopifnot(max(abs(colMeans(car_z))) < 1e-10,
          max(abs(apply(car_z, 2, sd) - 1)) < 1e-10)
car_distances <- dist(car_z, method = "euclidean")
```

```r
car_trees <- lapply(c("single", "complete", "average"), function(method) {
  hclust(car_distances, method = method)
})
names(car_trees) <- c("Single", "Complete", "Average")
```

```r
old_par <- par(mfrow = c(3, 1), mar = c(7, 4, 2, 1))
for (method in names(car_trees)) {
  plot(car_trees[[method]], main = paste(method, "linkage"),
       xlab = "", sub = "", cex = 0.65, hang = -1)
}
par(old_par)
```

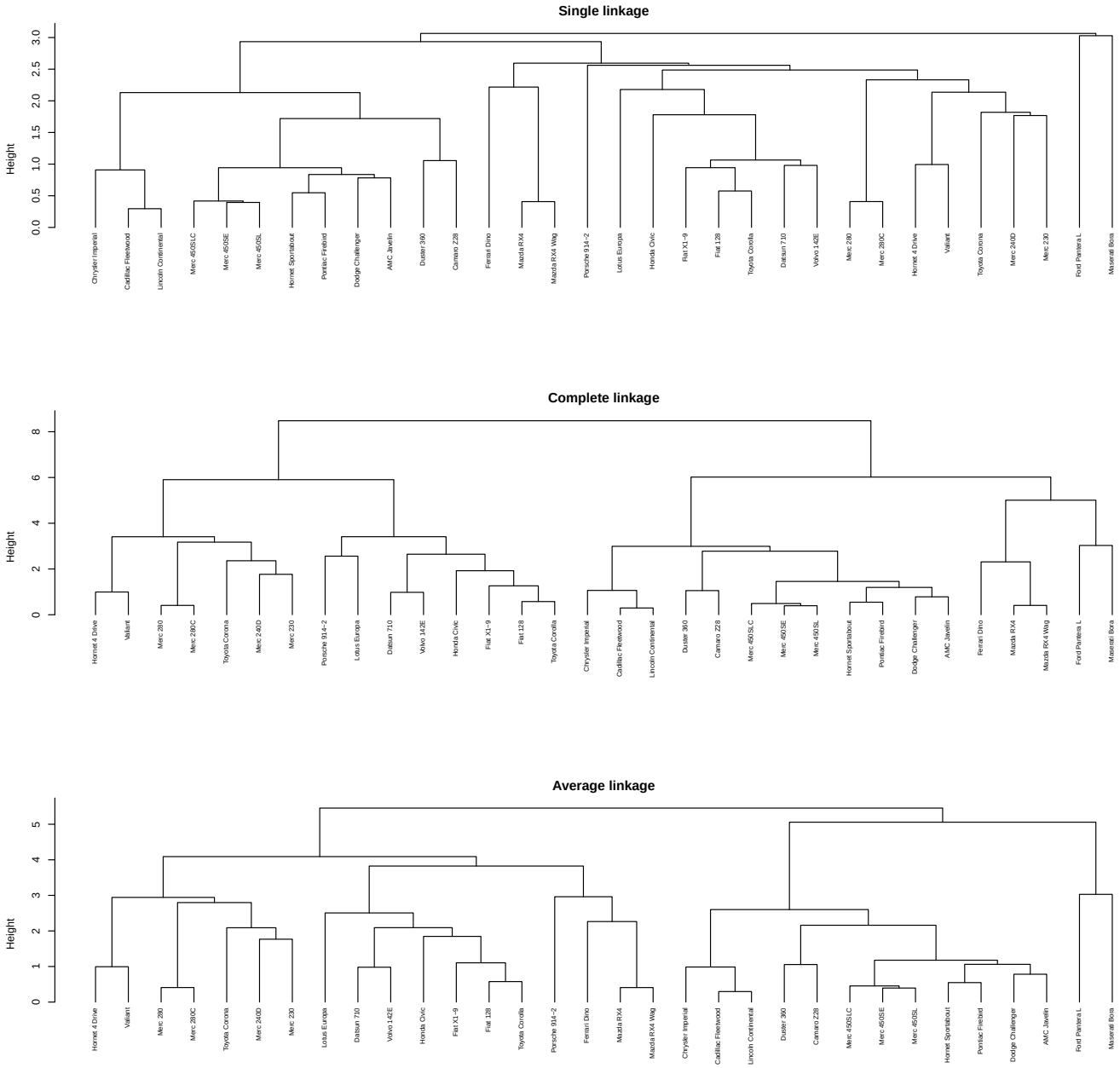


Figure 5: Hierarchical clustering of standardized car measurements using three linkage methods. Heights represent the linkage dissimilarity at each merge.

Single linkage shows chaining, with cars joining a growing group one at a time. Its final merges are close in height, making distinct groups harder to identify. **Complete linkage** has clearer branches, separating the large eight-cylinder automatics from a group containing the Mazda RX4 models, Ferrari Dino, Ford Pantera L, and Maserati Bora. **Average linkage** shows less chaining than single linkage, but places Ford Pantera L and Maserati Bora with the large eight-cylinder group instead. All three methods keep similar pairs together, including Mazda RX4/Mazda RX4 Wag and Cadillac Fleetwood/Lincoln Continental.

Part (b): Complete linkage with three clusters

I choose **complete linkage with three clusters** because the groups distinguish lighter, fuel-efficient cars, manual performance-oriented cars, and large eight-cylinder automatics. Cutting at height **5.96**, between merges at 5.90 and 6.02, gives these groups. The narrow gap means three clusters are not the only reasonable choice; the larger final jump also supports a coarser two-cluster solution.

```
car_cluster <- cutree(car_trees$Complete, k = 3)
stopifnot(identical(car_cluster, cutree(car_trees$Complete, h = 5.96)))
car_memberships <- data.frame(car = cars$car,
                              cluster = unname(car_cluster))
car_memberships <- car_memberships[
  order(car_memberships$cluster, car_memberships$car), ]
knitr::kable(car_memberships, row.names = FALSE,
             caption = "Complete-linkage cluster memberships")
```

Table 8: Complete-linkage cluster memberships

car	cluster
Ferrari Dino	1
Ford Pantera L	1
Maserati Bora	1
Mazda RX4	1
Mazda RX4 Wag	1
Datsun 710	2
Fiat 128	2
Fiat X1-9	2
Honda Civic	2
Hornet 4 Drive	2
Lotus Europa	2
Merc 230	2
Merc 240D	2
Merc 280	2
Merc 280C	2
Porsche 914-2	2
Toyota Corolla	2
Toyota Corona	2
Valiant	2
Volvo 142E	2
AMC Javelin	3
Cadillac Fleetwood	3
Camaro Z28	3
Chrysler Imperial	3
Dodge Challenger	3
Duster 360	3
Hornet Sportabout	3
Lincoln Continental	3
Merc 450SE	3
Merc 450SL	3
Merc 450SLC	3
Pontiac Firebird	3

```
car_means <- aggregate(car_data, by = list(cluster = car_cluster), FUN = mean)
car_means$n <- as.integer(table(car_cluster)[as.character(car_means$cluster)])
car_means <- car_means[, c("cluster", "n", names(car_data))]
```

```r
if (knitr::is_latex_output()) {
  print(knitr::kable(car_means[, c("cluster", "n", "mpg", "cyl", "disp", "hp", "drat")],
                     digits = 2, row.names = FALSE,
                     caption = "Cluster means: fuel economy and engine measurements"))
  print(knitr::kable(car_means[, c("cluster", "wt", "qsec", "vs", "am", "gear", "carb")],
                     digits = 2, row.names = FALSE,
                     caption = "Cluster means: weight, performance, and drivetrain"))
} else {
  knitr::kable(car_means, digits = 2, row.names = FALSE,
               caption = "Cluster means on the original variable scales")
}
```

Table 9: Cluster means: fuel economy and engine measurements

cluster	n	mpg	cyl	disp	hp	drat
1	5	18.50	6.80	223.40	198.80	3.84
2	15	24.65	4.53	131.65	91.33	3.90
3	12	15.05	8.00	357.62	194.17	3.12

Table 10: Cluster means: weight, performance, and drivetrain

cluster	wt	qsec	vs	am	gear	carb
1	3.00	15.62	0.00	1.00	4.60	5.20
2	2.58	19.16	0.93	0.53	3.93	1.80
3	4.10	17.14	0.00	0.00	3.00	3.08

```r
old_par <- par(mar = c(8, 4, 3, 1))
plot(car_trees$Complete, main = "Complete linkage: three clusters",
     xlab = "", sub = "", cex = 0.75, hang = -1)
abline(h = 5.96, lty = 2, col = "gray40")
rect.hclust(car_trees$Complete, k = 3,
            border = c("#0072B2", "#D55E00", "#009E73"))
par(old_par)
```

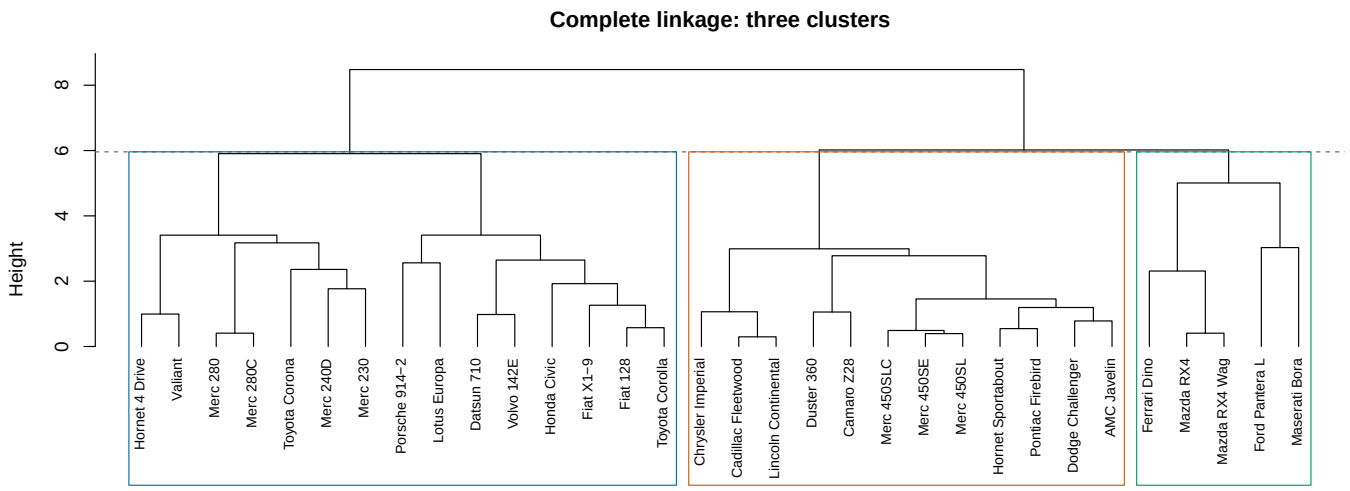


Figure 6: Complete-linkage dendrogram with the selected three clusters outlined and the cut height marked.

- **Cluster 1 (5 cars)** contains the Mazda RX4 models, Ferrari Dino, Ford Pantera L, and Maserati Bora. All have manual transmissions. They average **198.80 horsepower**, **18.50 mpg**, and **3.00 thousand pounds**, with the shortest mean quarter-mile time (**15.62 seconds**). These are relatively powerful, performance-oriented cars.
- **Cluster 2 (15 cars)** has the best mean fuel economy (**24.65 mpg**), lowest mean horsepower (**91.33**), smallest mean displacement (**131.65 cubic inches**), and lowest mean weight (**2.58 thousand pounds**). The group consists mostly of four-cylinder cars, with some six-cylinder cars. Its mean quarter-mile time is **19.16 seconds**, the longest of the three groups.
- **Cluster 3 (12 cars)** consists entirely of eight-cylinder cars with automatic transmissions and three forward gears. It has the largest mean displacement (**357.62 cubic inches**) and weight (**4.10 thousand pounds**), and the lowest mean fuel economy (**15.05 mpg**). Its mean horsepower is **194.17**, similar to cluster 1, but its mean quarter-mile time is longer (**17.14 seconds**).

In the table, `wt` is in thousands of pounds, `disp` is in cubic inches, and `qsec` is in seconds. The means of `am` and `vs` are the proportions with manual transmissions and straight engines, respectively.

Standardization prevents variables with large numerical ranges, such as displacement and horsepower, from dominating Euclidean distance. Each variable then contributes on the same standard-deviation scale despite its different units.